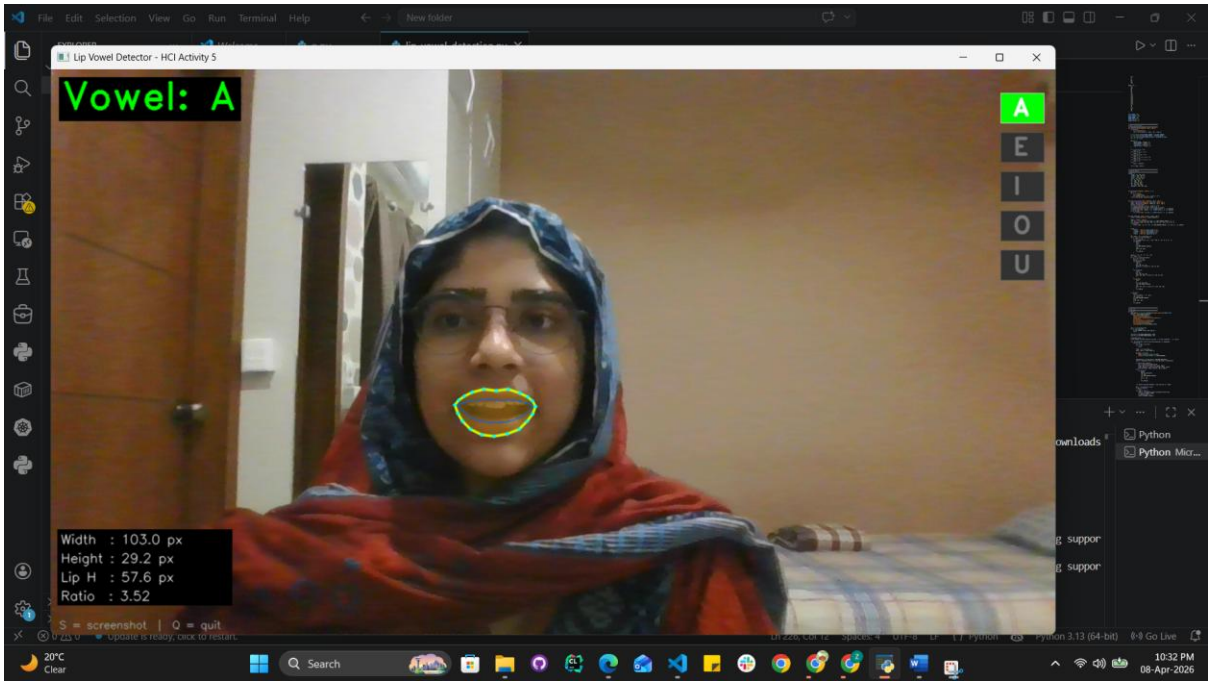
# CLASS ACTIVITY-5: LIP DETECTION AND VOWEL CLASSIFICATION USING MEDIAPIPE

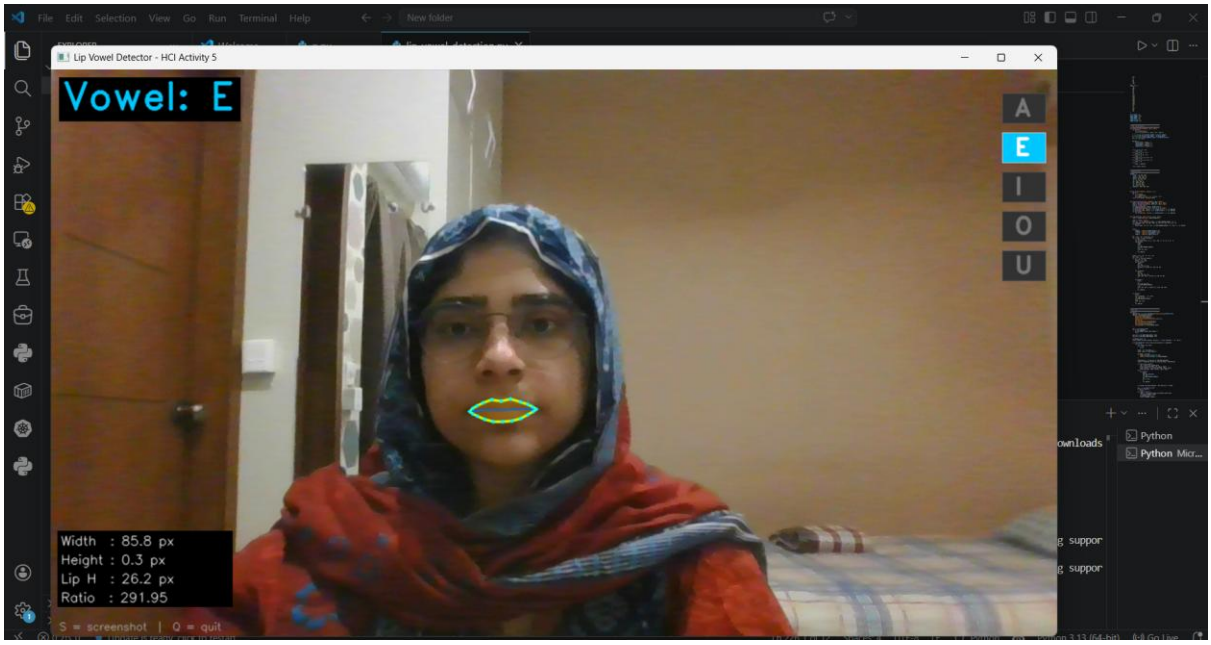
Zunaira Abdul Aziz

BSSE23058-A

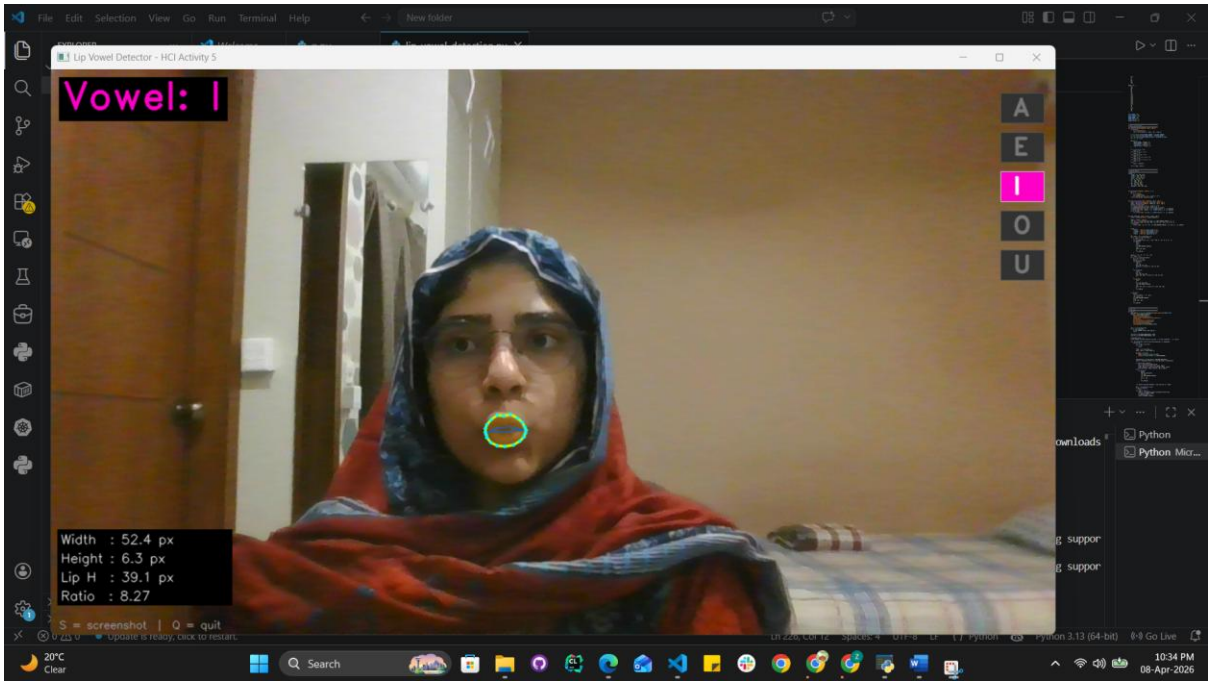
# VOWEL A:



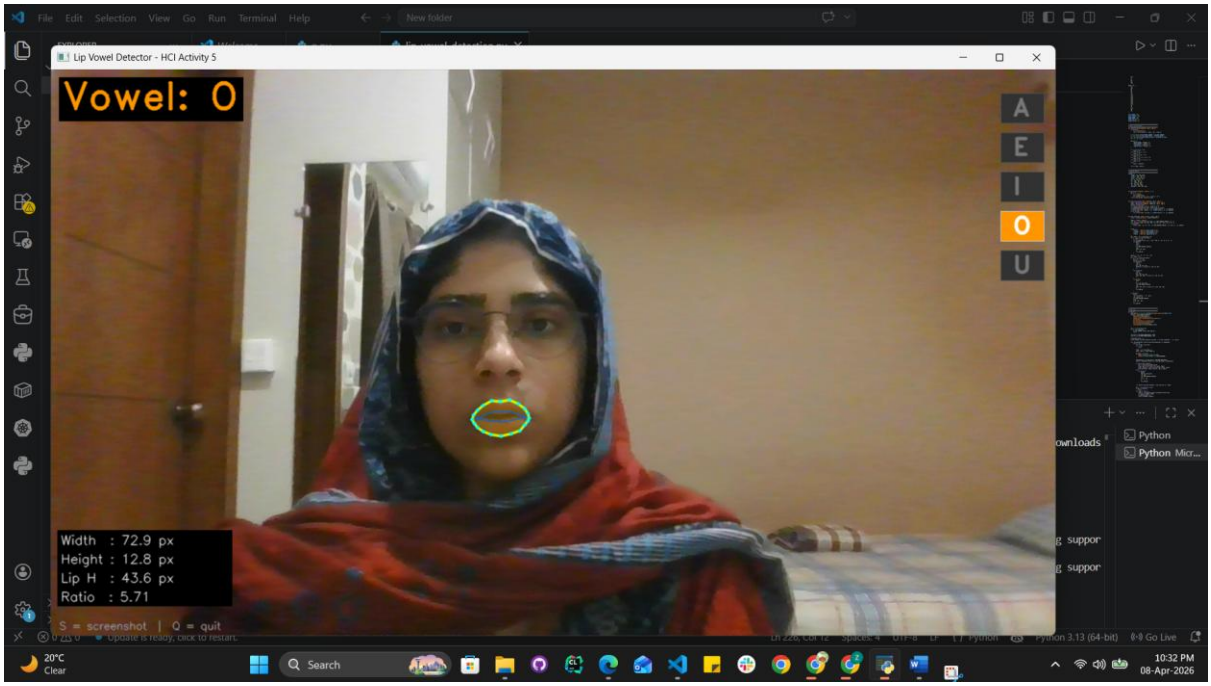

# VOWEL E:



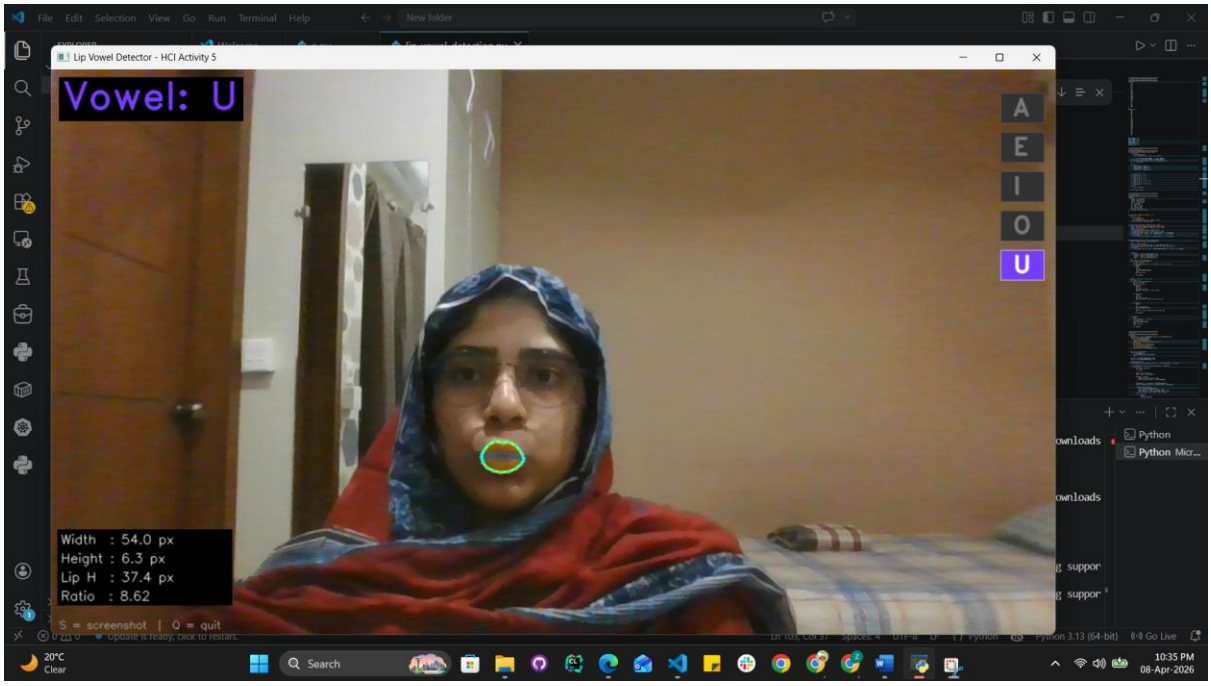
## VOWEL I:



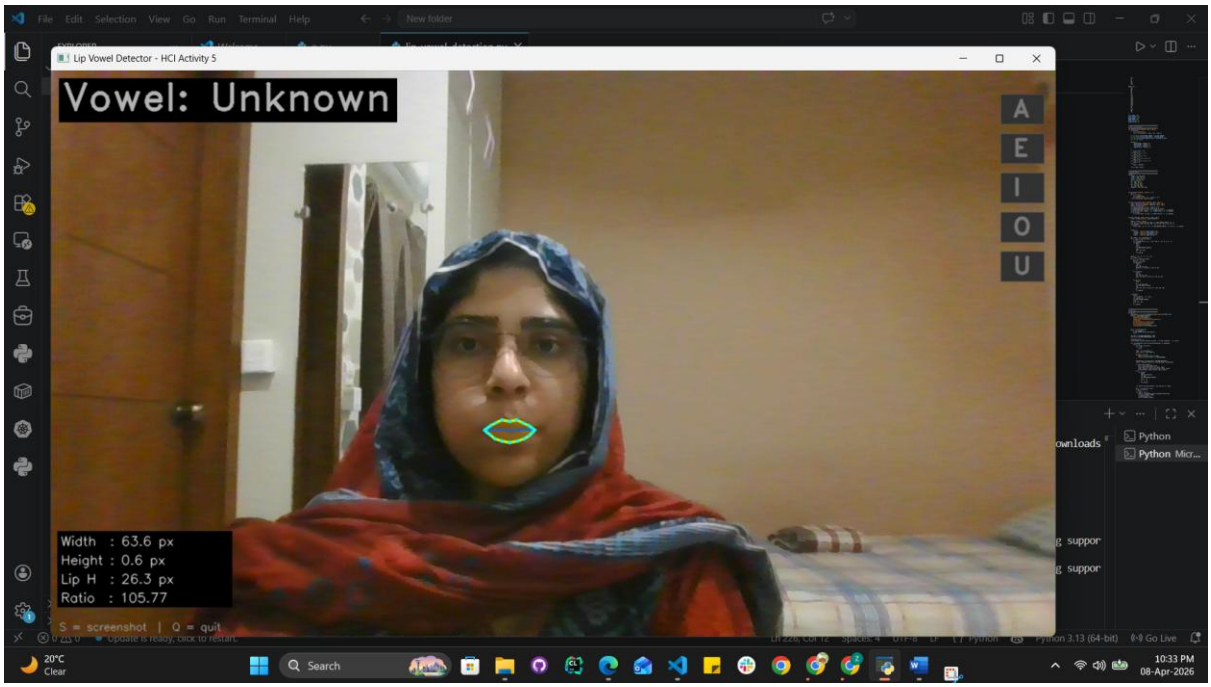

## VOWEL O:



## VOWEL U:




## UNKNOWN:




## CODE:

```
import cv2
import numpy as np
import mediapipe as mp
from mediapipe.tasks import python as mp_python
from mediapipe.tasks.python import vision
from mediapipe.tasks.python.vision import FaceLandmarker,
FaceLandmarkerOptions
```

```python
import urllib.request
import os

# ─────────────────────────────────────────────
# Download model if not present
# ─────────────────────────────────────────────
MODEL_PATH = "face_landmarker.task"
MODEL_URL  = "https://storage.googleapis.com/mediapipe-
models/face_landmarker/face_landmarker/float16/1/face_landmarker.task"

if not os.path.exists(MODEL_PATH):
    print("[INFO] Downloading face_landmarker.task model (~6 MB)...")
    urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)
    print("[INFO] Download complete.")

# ─────────────────────────────────────────────
# Landmark indices (MediaPipe 478-point mesh)
# ─────────────────────────────────────────────
OUTER_LIP = [61, 146, 91, 181, 84, 17, 314, 405, 321, 375,
             291, 409, 270, 269, 267, 0, 37, 39, 40, 185]
INNER_LIP = [78, 191, 80, 81, 82, 13, 312, 311, 310, 415,
             308, 324, 318, 402, 317, 14, 87, 178, 88, 95]

LEFT_CORNER   = 61
RIGHT_CORNER  = 291
UPPER_INNER   = 13
LOWER_INNER   = 14
UPPER_LIP_TOP = 0
LOWER_LIP_BOT = 17

# ─────────────────────────────────────────────
# Vowel classification
# ─────────────────────────────────────────────
def classify_vowel(landmarks, img_w, img_h):
    def pt(idx):
        lm = landmarks[idx]
        return np.array([lm.x * img_w, lm.y * img_h])

    w  = np.linalg.norm(pt(RIGHT_CORNER) - pt(LEFT_CORNER))
    h  = np.linalg.norm(pt(LOWER_INNER)  - pt(UPPER_INNER))
    lh = np.linalg.norm(pt(LOWER_LIP_BOT) - pt(UPPER_LIP_TOP))
    ar = w / (h + 1e-6)

    metrics = {
        "mouth_width":  round(w,  1),
        "mouth_height": round(h,  1),
        "lip_height":   round(lh, 1),
        "aspect_ratio": round(ar, 2),
```

```
    }

    if h > 18 and ar < 4.0:
        vowel = "A"
    elif w > 70 and ar > 6.0:
        vowel = "E"
    elif w < 55 and ar > 7.0:
        vowel = "I"
    elif 4.0 <= ar <= 6.5 and h > 8:
        vowel = "O"
    elif 4.5 <= ar <= 7.0 and h > 5:
        vowel = "U"
    else:
        vowel = "Unknown"

    return vowel, metrics

# ──────────────────────────────────────────────
# Drawing helpers
# ──────────────────────────────────────────────
COLORS = {
    "outer":   (0,   255, 200),
    "inner":   (255, 100,   0),
    "point":   (255, 255,   0),
    "fill":    (0,   180, 255),
    "A":       (0,   255,   0),
    "E":       (255, 200,   0),
    "I":       (200,   0, 255),
    "O":       (0,   150, 255),
    "U":       (255,  60, 120),
    "Unknown": (180, 180, 180),
}

def get_points(landmarks, indices, w, h):
    pts = []
    for i in indices:
        lm = landmarks[i]
        pts.append((int(lm.x * w), int(lm.y * h)))
    return np.array(pts, dtype=np.int32)

def draw_lip_overlay(frame, landmarks, img_w, img_h):
    outer = get_points(landmarks, OUTER_LIP, img_w, img_h)
    inner = get_points(landmarks, INNER_LIP, img_w, img_h)
    overlay = frame.copy()
    cv2.fillPoly(overlay, [outer], COLORS["fill"])
    cv2.addWeighted(overlay, 0.25, frame, 0.75, 0, frame)
    cv2.polylines(frame, [outer], True, COLORS["outer"], 2, cv2.LINE_AA)
    cv2.polylines(frame, [inner], True, COLORS["inner"], 1, cv2.LINE_AA)
```

```python
    for p in outer:
        cv2.circle(frame, tuple(p), 2, COLORS["point"], -1, cv2.LINE_AA)

def draw_hud(frame, vowel, metrics, img_w, img_h):
    color = COLORS.get(vowel, COLORS["Unknown"])

    label = f"Vowel: {vowel}"
    (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_DUPLEX, 1.6, 2)
    cv2.rectangle(frame, (10, 10), (20 + tw, 20 + th + 10), (0, 0, 0), -1)
    cv2.putText(frame, label, (15, 15 + th),
                cv2.FONT_HERSHEY_DUPLEX, 1.6, color, 2, cv2.LINE_AA)

    lines = [
        f"Width  : {metrics['mouth_width']} px",
        f"Height : {metrics['mouth_height']} px",
        f"Lip H  : {metrics['lip_height']} px",
        f"Ratio  : {metrics['aspect_ratio']}",
    ]
    y0 = img_h - 20 - len(lines) * 24
    for i, line in enumerate(lines):
        y = y0 + i * 24
        cv2.rectangle(frame, (8, y - 18), (230, y + 6), (0, 0, 0), -1)
        cv2.putText(frame, line, (12, y),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55,
                    (220, 220, 220), 1, cv2.LINE_AA)

    vowels = ["A", "E", "I", "O", "U"]
    bx = img_w - 70
    for vi, v in enumerate(vowels):
        by = 30 + vi * 50
        active = (v == vowel)
        cv2.rectangle(frame, (bx, by), (bx + 55, by + 38),
                      COLORS[v] if active else (50, 50, 50), -1)
        cv2.rectangle(frame, (bx, by), (bx + 55, by + 38),
                      (200, 200, 200) if active else (80, 80, 80), 1)
        cv2.putText(frame, v, (bx + 16, by + 28),
                    cv2.FONT_HERSHEY_DUPLEX, 1.0,
                    (255, 255, 255) if active else (120, 120, 120),
                    2, cv2.LINE_AA)

    cv2.putText(frame, "S = screenshot  |  Q = quit",
                (10, img_h - 8), cv2.FONT_HERSHEY_SIMPLEX,
                0.45, (150, 150, 150), 1, cv2.LINE_AA)

# ──────────────────────────────────────────────
# Main loop
# ──────────────────────────────────────────────
def main():
```

```
    base_options = mp_python.BaseOptions(model_asset_path=MODEL_PATH)
    options = FaceLandmarkerOptions(
        base_options=base_options,
        output_face_blendshapes=False,
        output_facial_transformation_matrixes=False,
        num_faces=1,
        min_face_detection_confidence=0.6,
        min_face_presence_confidence=0.6,
        min_tracking_confidence=0.6,
        running_mode=vision.RunningMode.VIDEO,
    )

    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("[ERROR] Cannot open webcam.")
        return

    cap.set(cv2.CAP_PROP_FRAME_WIDTH,  1280)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

    screenshot_count = 0
    print("[INFO] Lip Vowel Detector started.  S = save screenshot  |  Q =
quit")

    with FaceLandmarker.create_from_options(options) as landmarker:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            frame = cv2.flip(frame, 1)
            img_h, img_w = frame.shape[:2]

            mp_image = mp.Image(
                image_format=mp.ImageFormat.SRGB,
                data=cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            )

            timestamp_ms = int(cap.get(cv2.CAP_PROP_POS_MSEC))
            result = landmarker.detect_for_video(mp_image, timestamp_ms)

            if result.face_landmarks:
                lms = result.face_landmarks[0]
                draw_lip_overlay(frame, lms, img_w, img_h)
                vowel, metrics = classify_vowel(lms, img_w, img_h)
                draw_hud(frame, vowel, metrics, img_w, img_h)
            else:
                cv2.putText(frame, "No face detected",
```

```
                            (30, 60), cv2.FONT_HERSHEY_SIMPLEX,
                            1.0, (0, 0, 255), 2, cv2.LINE_AA)

            cv2.imshow("Lip Vowel Detector - HCI Activity 5", frame)

            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'):
                break
            elif key == ord('s'):
                fname = f"screenshot_vowel_{screenshot_count}.png"
                cv2.imwrite(fname, frame)
                screenshot_count += 1
                print(f"[SAVED] {fname}")

    cap.release()
    cv2.destroyAllWindows()
    print("[INFO] Session ended.")

if __name__ == "__main__":
    main()
```